RIPHAH INTERNATIONAL UNIVERSITY, LAHORE

FINAL YEAR PROJECTS (FYP) GUIDELINES

Product Development-Category Projects

Operational Guidelines and Governance Policy

Document Attribute	Details
Document Title	Operational Guidelines and Governance Policy for Product Development-Category Final Year Projects
Target Audience	Final Year Undergraduate Students, FYP Supervisors, Evaluation Panel Members, and relevant FYP Coordinators/Committees
Academic Discipline	Computing Disciplines including Computer Science, Software Engineering, Data Science, Artificial Intelligence, Cybersecurity, and related fields
Document Version	Version 1.0
Effective Date	Fall 2026 Onwards
Issuing Authority	Riphah School of Computing & Innovation (RSCI), Riphah International University, Lahore
Document Status	Approved Operational Policy & Student Directive

Policy precedence

This document shall be read together with applicable Riphah International University regulations, FYP procedures, academic-integrity requirements, HEC policies/frameworks, research-ethics requirements, intellectual-property provisions, data-protection obligations, and applicable law. Where a higher-authority requirement conflicts with this document, the higher-authority requirement shall prevail.

Version 1.0 — Fall 2026 Onwards

Contents

Essential (Must Do) 2
1. Problem Identification 2
2. Existing System and Market Analysis 2
3. Proposing the Solution 2
4. Defining Scope 3
5. Requirements Engineering & System Scope 3
6. Technical Architecture & Implementation 4
7. Special Guidelines for IoT & Embedded Systems Projects 6

Recommended (Should Do) 6

Optional (Nice to Have) 7

Universal Dataset Governance & Data Disclaimer Policy 7

AI Tool Usage Policy: 7

Self-Check for Teams and Supervisors 7

Essential (Must Do)

1. Problem Identification

- **Start from a user and a context.** Name who has the problem, where it happens and how often. "Students" is too broad; "students ordering food at Riphah Lahore cafeterias during the 30-minute break" is specific.
- **Back it with evidence.** Use at least one source: a user survey (state the sample size), interviews, observation, SME consultation, incident or usage data, or credible published sources. Opinions and "we think" are not evidence.
- **Describe the current workaround and its cost.** Say what users do today and what it costs them in time, money, errors, safety or missed opportunity.
- **Find the root cause.** Use a method such as 5 Whys or a cause-and-effect diagram, and state the one core problem your product addresses. Avoid a bundle of loosely related problems.
- **Keep problem and solution separate.** "There is no app for X" is a missing solution, not a problem. Write the problem so that it would still be true if your product did not exist.
- **Define how success is measured.** Give at least one measurable indicator (e.g. reduce waiting time from 12 to under 4 minutes).

2. Existing System and Market Analysis

- **Survey existing solutions.** List every closely related product by features and target audience: commercial products, open-source tools, and earlier FYPs or academic prototypes. Use credible, recent sources and cite them.
- **Competitive analysis matrix.** Compare your product with at least 3 alternatives on the same criteria: key features, platform, pricing or licence, target users, strengths, limitations and local fit (language, payment, regulations, connectivity).
- **Target market and users.** Define the primary user segment and a rough size, even at order-of-magnitude level (e.g. about 6,000 students at the home campus, about 1.2 million university students nationally). State where and how the first users will be reached. Full market sizing (TAM, SAM, SOM), a revenue model and cost drivers are required for Innovative and Entrepreneurial projects and are encouraged for all others.
- **Evidence of demand.** Show that people would use it: survey results, interview quotes, pilot interest or letters of support from a stakeholder.

3. Proposing the Solution

- **Objectives first.** State 3 to 5 SMART objectives (specific, measurable, achievable, relevant, time-bound) that directly answer the problem. Features come after objectives.

- **Describe the approach.** Explain what the product does, for whom, and how the core modules work together, in plain language, before the technical detail.
- **Show traceability.** Each objective maps to features, each feature to numbered FRs and user stories, and each requirement to a test. Use a table like the one below.
- **Justify choices.** Explain why this platform, architecture and technology stack, and name at least one alternative you considered and rejected.
- **Demonstrate technical depth.** Identify the hardest technical problem in the project and how you will solve it. This is what separates the product from a tutorial-level application.
- **Prove feasibility.** Confirm that the team has the skills, time, data, hardware and tools needed, and note the main risks and fallbacks.

4. Defining Scope

- **Describe each module in 5 to 6 lines.** Cover its purpose, main functions, users and inputs and outputs.
- **State what is in and what is out.** List included features and the features deliberately excluded or deferred to future work. Exclusions show judgement and protect the team from scope creep.
- **Core requirements.** Summarise 3 to 5 core functional requirements and 2 to 3 user stories.
- **Prioritise.** Rank features as Must, Should or Could. Build and finish the Must items first.
- **Record assumptions, dependencies and constraints** (platform, data access, hardware, third-party services, time).
- **Favour depth over breadth.** A few fully working, well-tested core modules beat many shallow ones.
- **Control changes.** Any change to scope after approval is agreed with the supervisor and logged in the Change Record.

5. Requirements Engineering & System Scope

- **Traceable Functional Requirements (FRs):** Each FR has a unique tag (e.g. REQ-SF1-1), belongs to a named system feature with a High/Medium/Low priority, and maps to a verified end-user need. Include the stimulus/response sequence for each feature and the expected behaviour on invalid input. Specify functional requirements that map directly to verified end-user needs.
- **Standard User Stories:** Write at least 2 to 3 primary stories in the form: "As a [user], I want to [action], so that [benefit]." Each story links to one or more FRs.
- **Non-Functional Requirements (NFRs):** Define explicit operational criteria, including system performance, response latency, security protocols, and availability

targets. State measurable criteria for performance and response latency, security (authentication, data protection), availability, usability and, where relevant, safety. Use numbers (e.g. "search returns in under 2 seconds for 95% of requests"), not adjectives.

- **User classes and operating environment.** Identify each user class, its privileges and technical level, and the platform the product runs on (Android, Web, Windows Desktop, Embedded). Architecture must follow the platform and the functional requirements.
- **Competitive Analysis Matrix:** Compare the product against commercial and open-source alternatives on features, target audience and limitations. Conclude with a gap analysis showing what your product adds.
- **SDLC Lifecycle & Feature Depth:** Follow the full Software Development Life Cycle (SDLC) to build a complete computing solution. Focus on high-impact core modules delivering technical complexity beyond basic API wrappers or tutorial-level software. Thin API wrappers and tutorial-level applications are not acceptable. State clearly what is in scope and what is excluded.
- **Scope & Feature Depth:** Focus on core, high-impact modules and deliver technical complexity beyond simple API wrappers or basic tutorial-level applications.
- **Platform & Domain Alignment:** Align project architecture with the target platform (e.g., Android, Web, Windows Desktop, Embedded Systems) based on functional requirements.
- **Use Case Analysis:** Provide a use case model (diagram) covering all primary actors and features.
- Provide fully dressed use case descriptions for the core features, including preconditions, main flow, alternate flows and postconditions.

6. Technical Architecture & Implementation

- **Layered Multi-Tier Architecture:** Provide a detailed architectural diagram showing component interactions across presentation layers, business logic, backend frameworks, database schemas, and external APIs.
- **Design models.** Includes the domain model, ER diagram with data dictionary, class diagram, sequence diagrams, operation contracts, activity diagram, state transition diagram, component diagram and deployment diagram. A data flow diagram (Levels 0 and 1) is required only if a structured approach is used.
- **Detailed System Architecture:** Update and finalize architectural diagrams illustrating component interactions, backend frameworks, data flow, and APIs.
- **AI/ML Technical Depth:** State exact AI/ML/NLP model architectures, frameworks, specific versions (e.g., YOLOv8, HuggingFace Transformers), parameter sizing, fine-

tuning strategies, and evaluation pipelines rather than relying on black-box commercial wrappers.

- **UI/UX Mockups:** Include low-fidelity wireframes or mockups for the top 2–3 key screens, detailing the intended design system, UI component libraries, and accessibility standards.
- **Implementation:** Show important control flows or pseudo-code for the core modules.
- List components, libraries, web services and stubs, with versions.
- State the deployment environment, tools and techniques, coding standards followed, and the version control practice (repository, branching, commit discipline). Every member must have visible commits.
- **Risk & Integration Testing:** Detail potential technical risks, fallback/recovery options, unit testing suites, and system integration strategies.
- **Risk register.** List technical risks with a fallback or recovery option for each, in the Proposal.
- **Test coverage.** The Final Report documents use case testing, equivalence partitioning, boundary value analysis, data flow testing, unit testing, integration testing, performance testing and stress testing. Mark any technique that does not apply and say why.
- Link test cases back to the FRs and NFRs so requirement coverage is visible. Report results with pass/fail counts and the measured performance against the NFR targets.

Required Deliverables

- **Core Documentation & Code Artifacts:** Software Requirements Specification (SRS), System Design Document (SDD), Test Plan and Test Report, User Manual, Technical Documentation, Final Project Report, and Source Code Repository.

Project Plan

- **Work Breakdown Structure.** Cover project management, documentation, and every system layer (development environment, presentation, business logic, data management, and physical layer where applicable).
- **Roles and Responsibility Matrix.** State who does what; every member owns identifiable technical work.
- **Gantt Chart.** Built in a standard project management tool, spanning enrolment to completion. Label every milestone and task with a title and assignee, break work into tasks of about one week or less, and link dependent tasks.

7. Special Guidelines for IoT & Embedded Systems Projects

These apply in addition to Section 3. They belong in the Proposal under System Architectural Design and Implementation Tools and Techniques, and in SRS Sections 2.3.2 (Hardware Interfaces), 2.3.4 (Communications Interfaces) and 2.5 (Performance, Safety).

- **Hardware Bill of Materials (BOM).** An itemised table with exact model or part numbers (e.g. ESP32-WROOM-32, not "microcontroller"), unit cost, quantity, total hardware budget and sourcing or availability notes (local supplier, import lead time). Cover the controller, sensors, actuators, communication modules and power supply.
- **Communication protocols and payloads.** Name and justify each protocol (MQTT, BLE, LoRa, HTTP/REST, Zigbee). Define the JSON payload structure, power constraints, battery life target, duty cycle and the processing split between edge device and cloud.
- **Environmental and hardware resilience.** State the operating conditions (e.g. IP65 housing for dust and moisture, temperature range). Define hardware bench-testing and field testing separately from software testing, and describe offline caching and failure fallback behaviour when connectivity or a sensor fails.
- **Hardware and sensor integration.** Explain hardware selection, latency handling and alternative sensor options (moved here from Recommended because the BOM and Final Report testing depend on it).
- **Final Report additions.** Include wiring and circuit diagrams in Chapter 4, hardware test results in Chapter 6, and assembly and calibration steps in the Administrator Manual (Appendix B).

Recommended (Should Do)

- **Evidence-based requirements.** Base requirements on user surveys, interviews with subject-matter experts, or domain standards, and cite the sample size and findings in the Proposal and Chapter 1.
- **Team technical coordination.** All members understand the whole system, the tools used and the demonstration flow, and can answer questions outside their own module.
- **Continuation projects.** If the project extends an earlier FYP or an existing product, the Introduction must identify the prior work with a citation, its current functional state, confirmed access to its code, and a list of existing versus new capability. The new work must be substantial enough to stand as an independent FYP, with a plan for safe integration and rollback.
- **Faculty feedback log.** Record every faculty-proposed change and the supervisor's decision in the List of Faculty Proposed Changes, and update the Change Record version history with each revision.

Optional (Nice to Have)

- Deploy a live hosted MVP/prototype on public cloud platforms (e.g., AWS, Vercel, Play Store beta).
- Integrate automated CI/CD deployment pipelines and automated unit testing suites.

Universal Dataset Governance & Data Disclaimer Policy

(Applies to all Final Year Projects across all categories using datasets, data collection, or processing)

- **Data Anonymization:** All Personally Identifiable Information (PII)—including names, contact details, email addresses, national ID numbers (CNIC), location metadata, and facial images—must be thoroughly masked, scrubbed, or anonymized prior to model training, processing, or repository submission.
- **Compliance with Data Laws:** Datasets must strictly comply with local and international privacy laws (e.g., Pakistan Prevention of Electronic Crimes Act - PECA, GDPR). Unauthorized scraping or scraping from restricted/private portals without written consent is strictly prohibited.
- **Dataset Sourcing & Licensing:** Ensure all external datasets possess clear open-source or academic licenses (e.g., MIT, CC BY 4.0). Any domain-specific data (e.g., medical, legal, financial) collected directly from users requires institutional review, formal consent, or SME authorization.
- **Regional Context & Origin:** Datasets must be context-relevant and address local/origin-oriented requirements (e.g., Pakistani demographic or localized domain data) where applicable. Explicitly detail how data collection and availability challenges will be resolved.

AI Tool Usage Policy:

- No AI usage/assistance in core functionalities and documentation
- AI application through APIs is allowed

Please consult with your respective supervisors to address these points in your next documentation revision.

Self-Check for Teams and Supervisors

Tick each item before the stage submission. Items marked "Final" apply only to the Final Report.

- ☐ FRs are numbered, prioritised, traceable to user needs, with 2 to 3 user stories
- ☐ NFRs have measurable targets
- ☐ Competitive analysis matrix and gap analysis are included
- ☐ Scope states depth of core modules and what is excluded
- ☐ Architecture diagram covers all layers and is up to date
- ☐ 2 to 3 key screen wireframes (Final: actual screens) with design system and accessibility notes
- ☐ AI/ML models, versions and evaluation pipeline stated (if applicable)
- ☐ Use case model and fully dressed use cases are complete
- ☐ Design models are present and consistent with the code
- ☐ Risk register with fallbacks is included
- ☐ WBS, Roles & Responsibility Matrix and Gantt Chart are complete
- ☐ Repository link with commits from every member
- ☐ Final: Cover all test types, with results mapped to requirements
- ☐ Final: Include critical review, lessons learnt and future work
- ☐ Final: User Manual, Administrator Manual and promotional material are attached
- ☐ IoT only: BOM, protocols and payloads, environmental conditions and fallback behaviour
- ☐ Dataset, licences cited, consent documented
- ☐ AI policy respected: no AI help in core functionality or documentation
- ☐ Template formatting, captions, references and Change Record updated; placeholders removed